Exercise Sheet 4: ODEs and dynamical systems

○ Self-study † Class discussion

Class discussion: Questions 3, 4, 5.

Use Python 3 with NumPy, SciPy and Matplotlib. No data files are needed. Use Class 6 and its try-it notebook as references. Label plot axes and legends, and check solver success before interpreting the results.

Interactive 3D plotting (optional)

Matplotlib draws the graph; its display backend determines where the graph appears and whether you can interact with it. With an interactive display, drag a 3D plot to rotate it and use the toolbar to zoom. Opening saved notebook outputs alone gives static pictures: rerun the plotting cells after enabling interaction. Rotation is an extra aid for exploring 3D plots.

What are PyQt and PySide6? They are alternative Python interfaces to Qt, a toolkit for creating application windows and handling mouse and keyboard input. Matplotlib can use either to open a separate interactive plot window. You do not need to write Qt code or install both. The notebook route below uses ipympl instead of requiring a separate Qt window.

Jupyter Notebook/Lab or VS Code notebooks. Install ipympl in the same Python environment as the notebook kernel. You can use this terminal command (use `python` instead of `python3` if that is your Python command):

```
python3 -m pip install ipympl
```

Alternatively, run `%pip install ipympl` in a notebook cell. Restart the kernel, then put `%matplotlib widget` at the top of the imports cell, before `import matplotlib.pyplot as plt`. Rerun the notebook. The interactive figure appears inside it. If Notebook itself is not installed, install it with `python3 -m pip install notebook`, then launch it with `python3 -m notebook`.

Normal Python scripts: terminal or text editor. If you do not already have a compatible Qt package, install PySide6 in the environment used to run the script:

```
python3 -m pip install PySide6
```

Put this at the start of your `.py` file. `QtAgg` selects Matplotlib's separate Qt plotting window:

```
import matplotlib
matplotlib.use("QtAgg")
import matplotlib.pyplot as plt
```

Keep your ODE and plotting code after these imports and finish with `plt.show()`. Run the saved file from the terminal, for example `python3 my_solution.py`. The window opens on your computer and lets you rotate the 3D plot.

PyCharm, VS Code or another IDE running a `.py` script. Use the script setup above and install the package in the IDE's selected Python environment. In PyCharm, if a static plot appears inside the IDE, disable "Show plots in tool window" in Settings (search for that option) and rerun the script. For a notebook inside an IDE, widget support depends on the IDE; if it remains static, open the notebook in Jupyter in your browser, or run a `.py` script in a separate Qt window.

Spyder or terminal IPython. With Qt support installed, run `%matplotlib qt` in the IPython console before plotting. In Spyder, you can instead select a Qt graphics backend under Preferences, IPython console, Graphics, then restart the console. Commands starting with `%` are for notebook cells or IPython consoles; do not put them in ordinary `.py` files.

Question 1 - Linear dynamics ◦

Consider the linear system

$$\begin{aligned} \dot{x} &= -x + 2y, \\ \dot{y} &= -2y, \end{aligned} \qquad (x(0), y(0)) = (1, 1).$$

Write the state as $z = (x, y)^T$.

(a) Write the system as $\dot{z} = Az$. Find its equilibrium and use `np.linalg.eigvals(A)` to predict its stability. Can a component increase initially even when every eigenvalue has negative real part? Check the signs of $\dot{x}(0)$ and $\dot{y}(0)$ before plotting.

(b) Define an appropriate function to pass to the ODE solver. Use `solve_ivp` on $0 \leq t \leq 6$, with `t_eval=np.linspace(0,6,301)`, `rtol=1e-6` and `atol=1e-9`. Check `sol.success` and `sol.message`. What are the shapes of `sol.t` and `sol.y`? Which entries contain the initial state and the final state?

(c) Plot $x(t)$ and $y(t)$ together, with a legend. In a separate plot draw the phase trajectory $(x(t), y(t))$, marking the start and the equilibrium. Compare the plots with your predictions in (a).

(d) The exact solution is

$$\begin{aligned} x(t) &= 3e^{-t} - 2e^{-2t}, \\ y(t) &= e^{-2t}. \end{aligned}$$

Check that it satisfies the initial condition and the differential equations. Evaluate both the exact and computed states at $t = 0.5, 2, 5$ and print their absolute differences. Use `t_eval=[0.5,2.,5.]` in a second solve; these values are times, not column indices.

(e) Repeat the original solve with `dense_output=True`. Evaluate `sol.sol([0.37,1.23,4.56])` and compare with the exact formula. Explain how this differs from simply indexing `sol.y`.

Question 2 - Nonlinear equilibria ◦

A particle moves between two preferred positions. Let x be position and v velocity. A simple damped model is

$$\dot{x} = v,$$
$$\dot{v} = x - x^3 - 0.4v.$$

(a) Find all equilibria by setting both derivatives to zero. Derive the Jacobian $J(x, v)$. Evaluate it at each equilibrium, compute its eigenvalues and classify the equilibrium using their real parts. State whether the prediction describes behaviour near that equilibrium or every possible initial state.

(b) Write `motion(t, state)` and solve on $0 \leq t \leq 40$ from each initial state $(0.05, 0)$, $(-0.05, 0)$ and $(1.1, 0)$. Use 2001 saved times and `rtol=1e-7`, `atol=1e-9`. Before running, predict which equilibrium each trajectory might approach. Plot all three $x(t)$ curves with labels, and draw their phase trajectories with the equilibria marked.

(c) For the two initial states near the origin, evaluate the state at $t = 5, 10, 20, 40$ using `t_eval` or dense output. Print a small table and compare the final distance from each of the three equilibria. Do the two nearby initial states have the same long-term outcome?

(d) Construct a vector field on $-1.6 \leq x \leq 1.6$, $-1.2 \leq v \leq 1.2$ using `np.meshgrid`. Plot it with `quiver` and overlay the trajectories. Use `angles="xy", scale_units="xy"` and an explicit `scale` of your choice. Explain why the arrows represent derivatives rather than completed integration steps.

Question 3 - Steps and accuracy †

Consider the initial-value problem

$$\dot{u} = -2u, \qquad u(0) = 1, \qquad u(t) = e^{-2t}.$$

The last expression is the exact solution. Here you can distinguish a numerical error from a feature of the differential equation.

(a) Euler's method uses

$$u_{n+1} = u_n + hf(t_n, u_n).$$

For this equation, show that $u_{n+1} = (1-2h)u_n$. Starting from $u_0 = 1$, calculate the first three updates for $h = 0.25$, 0.75 and 1.25. Which sequences decay, alternate in sign, or grow in magnitude? The exact solution is positive and decays.

(b) Implement Euler's method with a loop. For each step size in (a), use `n_steps=int(round(15/h))` and `times=h*np.arange(n_steps+1)` so the endpoint is 15. Plot the three approximations alongside the exact solution. Use separate panels if the growing numerical solution obscures the others. What condition on $h > 0$ gives $|1 - 2h| < 1$?

(c) Solve the same initial-value problem with `solve_ivp` and `method="RK45"`, requesting `t_eval=np.arange(0.,16.)`. Plot those saved values with the exact solution. Does requesting values one unit apart mean that RK45 takes internal steps of length one?

(d) Compare RK45 runs with `(rtol,atol)=(1e-3,1e-6)` and `(1e-8,1e-11)` on the same 301-point output grid from 0 to 15. For each run, report `sol.nfev` and the largest absolute error against the exact solution on that grid. Explain the roles of output spacing and tolerances. Do not interpret the tolerances as a guaranteed bound on the total accumulated error.

(e) A stiff system. Now consider two components that decay on very different time scales:

$$\begin{aligned} \dot{u} &= -u, \\ \dot{v} &= -1000v, \end{aligned} \qquad u(0) = v(0) = 1.$$

The exact solution is $u(t) = e^{-t}$, $v(t) = e^{-1000t}$. Which component changes rapidly, and which remains visible over a much longer time?

Solve on $0 \leq t \leq 10$ using both `method="RK45"` and `method="Radau"`, with `rtol=1e-6`, `atol=1e-9`. Use the same output times for both: combine `np.linspace(0,0.02,201)` and `np.linspace(0.02,10,501)[1:]` with `np.concatenate`. This gives extra saved points during the initial rapid decay without duplicating $t = 0.02$. Plot both computed components with the exact solutions, including a separate close-up of $v(t)$ for $0 \leq t \leq 0.02$.

For each method, report solver success, `sol.nfev` (derivative evaluations), and the largest absolute error in each component on the output grid. Radau is an implicit method designed to handle stiff problems. Which method uses fewer derivative evaluations? Relate your observation to the explanation of stiffness in the lecture.

Question 4 - Predator and prey †

Let $x(t)$ be a prey population and $y(t)$ a predator population, both in scaled units. Consider the family of models

$$\begin{aligned} \dot{x} &= x(1 - ax) - xy, \\ \dot{y} &= xy - dy, \end{aligned} \qquad a \geq 0, \quad d > 0.$$

The term xy describes encounters. The parameter a limits prey growth: $a = 0$ gives the classic Lotka–Volterra model, while $a > 0$ gives carrying capacity $1/a$ without predators. The parameter d is the predator death rate. Keep $d = 1$ in parts (a)-(c). A coexistence equilibrium is a fixed state where both populations are strictly positive and both derivatives are zero: both populations remain constant, and neither species disappears.

(a) Start with $a = 0$. Find the equilibria and the Jacobian, and compute the eigenvalues at the positive equilibrium. What can the lecture's Jacobian test conclude when their real parts are zero?

(b) Write `predator_prey(t, state, a, d)` and pass the parameters with `args=(a,d)`. With $a = 0$, solve from $(1.5, 0.5)$ on $0 \leq t \leq 30$, using 1501 saved times, `rtol=1e-8` and `atol=1e-10`. Plot both populations against time and draw the phase trajectory, marking the positive equilibrium. Which population peaks first? Does the trajectory appear to settle at the equilibrium or keep circling it?

(c) For general $a > 0$, find the prey-only equilibrium and the coexistence equilibrium (both populations positive). For which values of a is coexistence possible when $d = 1$? Evaluate the Jacobian at these equilibria and use its eigenvalues to predict where trajectories nearby converge.

(d) Explore a small grid of parameter choices: use $a = 0, 0.25, 1.25$ and $d = 0.5, 2$, giving six pairs. For each pair, solve from $(1.5, 0.5)$ on $0 \leq t \leq 300$, with 6001 saved times, `rtol=1e-8` and `atol=1e-10`. Arrange the phase plots in three rows (one per a) and two columns (one per d), labelling each panel with its parameter values. Use `np.meshgrid` and `quiver` to draw the vector field $(\dot{x}, \dot{y})$, then overlay the trajectory and mark the equilibria with nonnegative populations. Use `angles="xy", scale_units="xy"` and choose a `scale` that makes the arrows readable. Which pairs appear to give persistent oscillations, convergence to coexistence, or loss of predators? How do the fixed points move when a or d changes?

Question 5 - The Lorenz attractor †

The Lorenz system is a three-variable nonlinear model originally motivated by convection:

$$\begin{aligned}\dot{x} &= \sigma(y - x),\\ \dot{y} &= x(\rho - z) - y, \qquad \sigma = 10, \quad \rho = 28, \quad \beta = 8/3.\\ \dot{z} &= xy - \beta z,\end{aligned}$$

The coordinates are model variables. The aim is to explore the dynamics, practise 3D plotting and investigate what happens when the initial state changes slightly.

(a) Write `lorenz(t, state)` returning the three derivatives in the order x, y, z. Solve from $(1, 1, 1)$ on $0 \le t \le 40$, with `t_eval=np.linspace(0,40,8001)`, `rtol=1e-9`, `atol=1e-12`. Check success and inspect the returned array shapes. How many rows does `sol.y` have now?

(b) Plot the three coordinates against time. Then draw the trajectory in 3D using this plotting pattern, filling in the missing labels:

```
fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")
ax.plot(sol.y[0], sol.y[1], sol.y[2], linewidth=0.5)
# Add x, y and z labels, then show the figure.
```

Also plot the (x, z) projection in 2D. Rotate the 3D figure if your viewer supports it. Does one projection reveal everything visible in 3D?

(c) Repeat with initial state $(1 + 10^{-6}, 1, 1)$ and the same output times. Overlay the two $x(t)$ curves, first over $0 \le t \le 5$ and then over the full interval. Plot the distance between the two states against time using an ordinary line plot:

$$d(t) = \|z_1(t) - z_2(t)\|_2.$$

Here z_1, z_2 denote the full three-component states, not just their third coordinates. Compute the distances with `np.linalg.norm(sol1.y-sol2.y, axis=0)` and plot them with `plt.plot(sol1.t, distances)`. How small is the initial distance? Do the two trajectories remain close throughout the simulation? Describe what you observe. This experiment introduces sensitivity to initial conditions: a very small change in the starting state can eventually lead to noticeably different trajectories.

(d) As an accuracy check, solve again from the original initial state with tighter tolerances `rtol=1e-11`, `atol=1e-13`. Compare this trajectory with the original run, on both an early interval and the full interval. How does the difference compare with the initial-condition experiment in (c)? In a system sensitive to initial conditions, small numerical errors can also grow over time. What does this suggest about predicting one precise trajectory far into the future?

References: SciPy: solve_ivp; ACME: predator–prey models; Lorenz equations.